🐧 🔥 UBUNTU FULL PRACTICAL (ERROR-FREE)

🧩 STEP 1: Install Git

```
sudo apt update
sudo apt install git -y
```

Check:

```
git --version
```

🧩 STEP 2: Configure Git

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

🧩 STEP 3: Create Project

```
mkdir data-science-project-2
cd data-science-project-2
git init
```

🧩 STEP 4: Connect to GitHub

👉 Create repo on GitHub first

```
git remote add origin https://github.com/your-username/data-science-project-2.git
git branch -M main
```

🧩 STEP 5: Create program1.py (Even/Odd)

```
nano program1.py
```

Paste:

```
num = int(input("Enter a number: "))

if num % 2 == 0:
    print("Number is Even")
else:
    print("Number is Odd")
```

Save:
👉 CTRL + X → Y → ENTER

🧩 STEP 6: Commit & Push

```
git add .
git commit -m "Added program1 - even odd"
git push -u origin main
```

🧩 STEP 7: Create Branch

```
git checkout -b GIT-EXAM
```

🧩 STEP 8: Create program2.py (Reverse String)

```
nano program2.py
```

Paste:

```
text = input("Enter a string: ")
print("Reversed string is:", text[::-1])
```

Save again

🧩 STEP 9: Commit & Push

```
git add .
git commit -m "Added program2 - reverse string"
git push -u origin GIT-EXAM
```

🧩 STEP 10: Show Difference

```
git diff main GIT-EXAM
```

🧩 STEP 11: Create Pull Request (IMPORTANT)

👉 Go to GitHub

1. Open repo
2. Go to Pull Requests
3. Click New Pull Request
4. Select:
    - base → main

    - compare → GIT-EXAM
5. Click Create Pull Request
6. Click Merge Pull Request

⚠️ ZERO ERROR FIXES (IMPORTANT)

❌ Authentication Error

👉 Use Personal Access Token instead of password

❌ "nothing to commit"

```
git status
```

👉 Make sure file is modified

❌ No Pull Request option

👉 Fix:

```
git push origin GIT-EXAM
```

OR modify file and push again

❌ Permission denied

```
sudo chown -R $USER:$USER .
```

🧠 FINAL UNDERSTANDING

👉 Commands are same in Ubuntu + Git Bash

Task	Same Command?
git init	✅
git add	✅
git commit	✅
git push	✅

Task	Same Command?
git diff	✅

👉 Only file editing differs:

- Windows → Notepad
- Ubuntu → nano

🗣️ VIVA ANSWER

👉
"Yes, the same Git commands work in Ubuntu. Only installation and file handling differ slightly, but version control operations remain the same."

🧪 🔥 **WHAT THIS PRACTICAL IS (Simple Meaning)**

👉 You used **Git** and **GitHub** to:

- Create a project
- Track changes
- Work in a separate branch
- Add new feature
- Merge it using **Pull Request**

🧠 **WHAT YOU DID (Explain in 4 Lines)**

👉 Say this in exam:

"We created a Git repository, added a Python program to check even/odd, pushed it to GitHub, then created a new branch to add a reverse string program, and finally merged it into main using a pull request."

🔍 HOW IT WORKS (CORE LOGIC)

🔹 Step 1: Main Branch (main)

👉 You added:

- program1.py (Even/Odd)

👉 This is your **original project**

🔹 Step 2: New Branch (GIT-EXAM)

👉 You created branch and added:

- program2.py (Reverse String)

👉 This is like:
➡️ "Working on new feature safely"

🔹 Step 3: Difference Check

👉 Using:

```
git diff main GIT-EXAM
```

👉 Git shows:

- What is added
- What is changed

🔹 Step 4: Pull Request (MOST IMPORTANT)

👉 You merged:

- GIT-EXAM → main

👉 Using GitHub interface

🧠 WHY WE DO THIS

---

1. 🔒 Safe Development

👉 New work is done in branch
👉 Main project stays safe

---

2. 👀 Code Review

👉 Before merging:

- Code can be checked
- Errors can be avoided

---

3. 🔄 Controlled Integration

👉 Changes are merged properly
👉 No accidental damage

---

4. 👨‍💻 Real Industry Workflow

👉 Big companies use:

- Branch → PR → Review → Merge

⭐ IMPORTANCE (VERY IMPORTANT)

✓ 1. Industry Standard Practice

👉 Every company uses this workflow

✓ 2. Team Collaboration

👉 Many developers can:

- Work on different branches
- Merge safely

✓ 3. Version Control + Feature Control

👉 You can:

- Add features independently
- Merge when ready

✓ **4. DevOps Foundation**

👉 Pull Request is base for:

- CI/CD
- Code review pipelines

⚙️ **WORKING FLOW (VERY IMPORTANT)**

main → program1 (even/odd)
↓
create branch
↓
GIT-EXAM → program2 (reverse string)
↓
compare (git diff)
↓
pull request
↓
merge
↓
main updated

🧠 **CORE CONCEPTS**

🔹 **Branch**

👉 Separate version of code

🔹 **Commit**

👉 Save changes with message

🔹 **Push**

👉 Upload to GitHub

🔹 **Pull Request (KEY CONCEPT)**

👉 Request to merge one branch into another

🔹 **Merge**

👉 Combine code into main

🎯 **MODERATE LEVEL VIVA QUESTIONS**

❓ **What is Pull Request?**

👉
A pull request is a request to merge changes from one branch to another after review.

❓ **Why do we use Pull Request?**

👉
To review, discuss, and safely merge changes into the main branch.

❓ **Difference between merge and pull request?**

👉

- Merge → direct combining
- Pull Request → review + then merge

❓ **What is branch and why used?**

👉
A branch is used to develop features independently without affecting main code.

❓ **What is git diff?**

👉
It shows the difference between two branches or commits.

❓ **What happens if branches are identical?**

👉
No pull request can be created because there are no differences.

❓ **What is origin?**

👉
Origin is the default remote repository name (GitHub repo).

❓ **What is HEAD?**

👉
HEAD points to the current active branch or latest commit.

🔥 **SLIGHTLY ADVANCED (FOR EXTRA MARKS)**

❓ **What is code review?**

👉
Process of checking code before merging to improve quality and reduce errors.

❓ **What is merge conflict?**

👉
When two branches modify same part of code, Git asks to resolve conflict manually.

❓ **What is distributed version control?**

👉
Each user has a complete copy of the repository locally.

🧠 **ONE-LINE POWER ANSWER**

👉
"This practical demonstrates version control using Git and GitHub with branching and pull request workflow to safely develop and merge new features."

🏁 **FINAL UNDERSTANDING**

👉 This practical proves:

✓ You understand Git basics
✓ You can use branching
✓ You can manage versions
✓ You can merge using PR

👉 This is exactly how **real developers work**